

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Design Task

Code of Conduct:

I Hashini S / 192524035 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Hashini S

1. HYBRID AI RECOMMENDATION ENGINE DESIGN

1.1 Introduction

A hybrid AI recommendation engine combines **content-based filtering** and **collaborative filtering** to provide personalized recommendations. Content-based filtering recommends items similar to those a user has previously liked, while collaborative filtering recommends items based on the behaviour of similar users.

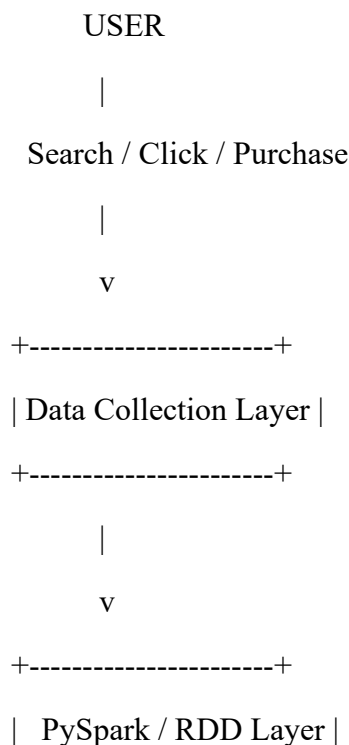
For an e-commerce platform, millions of users may interact with thousands or millions of products. Therefore, a distributed processing framework such as **PySpark** is required to process large-scale user-item interaction data efficiently.

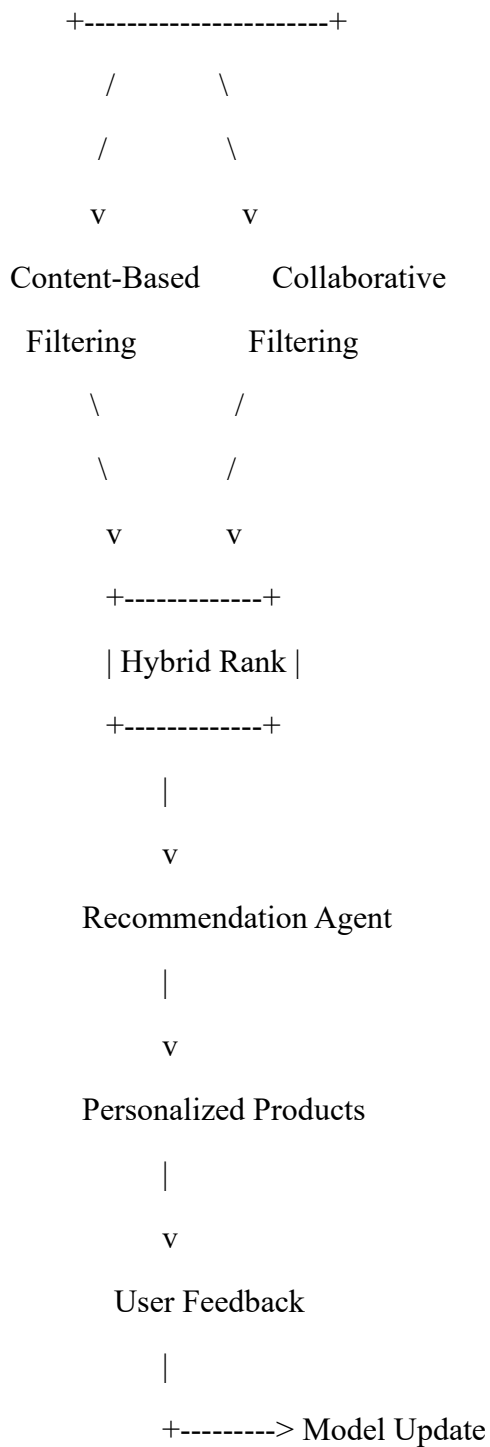
The proposed system uses **RDDs** to distribute data processing across multiple nodes and **intelligent agents** to dynamically adapt recommendations based on real-time user behaviour.

1.2 Objectives

- Provide personalized product recommendations.
- Process millions of user-item interactions.
- Combine content-based and collaborative filtering.
- Update recommendations according to real-time behaviour.
- Use intelligent agents for autonomous decision-making.
- Improve recommendation accuracy continuously.

1.3 System Architecture





1.4 Data Sources

The system can collect:

- User ID
- Product ID
- Product category
- Product description

- Product price
- User ratings
- Product views
- Search history
- Click history
- Cart activity
- Purchase history
- Time and session information

1.5 RDD-Based Data Processing

Suppose each interaction is represented as:

(user_id, product_id, rating)

The raw data can be loaded into an RDD:

```
interactions = sc.textFile("user_interactions.csv")
```

The records can be transformed:

```
records = interactions.map(lambda x: x.split(","))
```

Invalid records can be removed:

```
valid = records.filter(lambda x: len(x) == 3)
```

User-product relationships can be generated:

```
user_items = valid.map(
    lambda x: (x[0], (x[1], float(x[2])))
)
```

1.6 Content-Based Filtering

Content-based filtering compares product attributes.

For example:

User likes:

Laptop → Electronics → 16 GB RAM → Intel Processor

The system searches for products having similar characteristics.

Important product features may include:

- Category

- Brand
- Price
- Keywords
- Description
- Technical specifications

Similarity can be calculated using techniques such as **cosine similarity**.

1.7 Collaborative Filtering

Collaborative filtering uses behaviour from similar users.

Example:

User A → Laptop, Mouse, Keyboard

User B → Laptop, Mouse

Since User A and User B have similar behaviour, products preferred by User A can be recommended to User B.

PySpark can aggregate user-item interactions:

```
item_counts = valid.map(
    lambda x: ((x[0], x[1]), 1)
).reduceByKey(lambda a, b: a + b)
```

1.8 Hybrid Recommendation

The two approaches can be combined:

Hybrid Score =

$\alpha(\text{Content Score}) + \beta(\text{Collaborative Score})$

where:

- α = weight assigned to content-based filtering
- β = weight assigned to collaborative filtering

For example:

Content Score = 0.80

Collaborative Score = 0.70

$\alpha = 0.4$

$$\beta = 0.6$$

$$\begin{aligned}\text{Hybrid Score} &= (0.4 \times 0.80) + (0.6 \times 0.70) \\ &= 0.74\end{aligned}$$

Products with higher hybrid scores are ranked higher.

1.9 Intelligent Agents

The system can contain several agents.

Agent	Responsibility
User Behaviour Agent	Monitors clicks, searches and purchases
Recommendation Agent	Generates recommendations
Ranking Agent	Ranks candidate products
Feedback Agent	Analyses user responses
Learning Agent	Updates user preferences
Monitoring Agent	Detects system problems

1.10 Real-Time Adaptation

Suppose a user suddenly searches for **gaming laptops**.

The behaviour agent detects this event and updates the user's current-session preferences.

The recommendation agent can immediately increase the weight of:

- Gaming laptops
- GPUs
- Gaming accessories
- Mechanical keyboards

Thus, recommendations dynamically change according to user intent.

1.11 Advantages

- Highly personalized recommendations.
- Scalable processing.
- Better accuracy than a single filtering method.
- Real-time adaptation.

- Fault-tolerant distributed processing.
- Suitable for millions of users.

1.12 Conclusion

The proposed hybrid recommendation engine combines content-based and collaborative filtering using PySpark RDDs. Distributed processing allows huge user-item datasets to be processed efficiently, while intelligent agents continuously observe user behaviour and feedback. The resulting system is scalable, adaptive and capable of producing personalized recommendations in real time.

2. SMART DISASTER DETECTION SYSTEM

2.1 Introduction

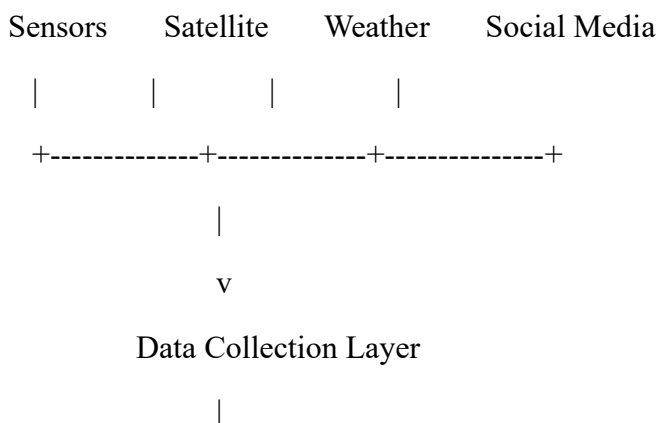
Natural disasters such as floods and earthquakes can cause significant damage when warnings are delayed. A Smart Disaster Detection System can combine information from **IoT sensors, satellite data, weather stations and social media** to identify disaster conditions early.

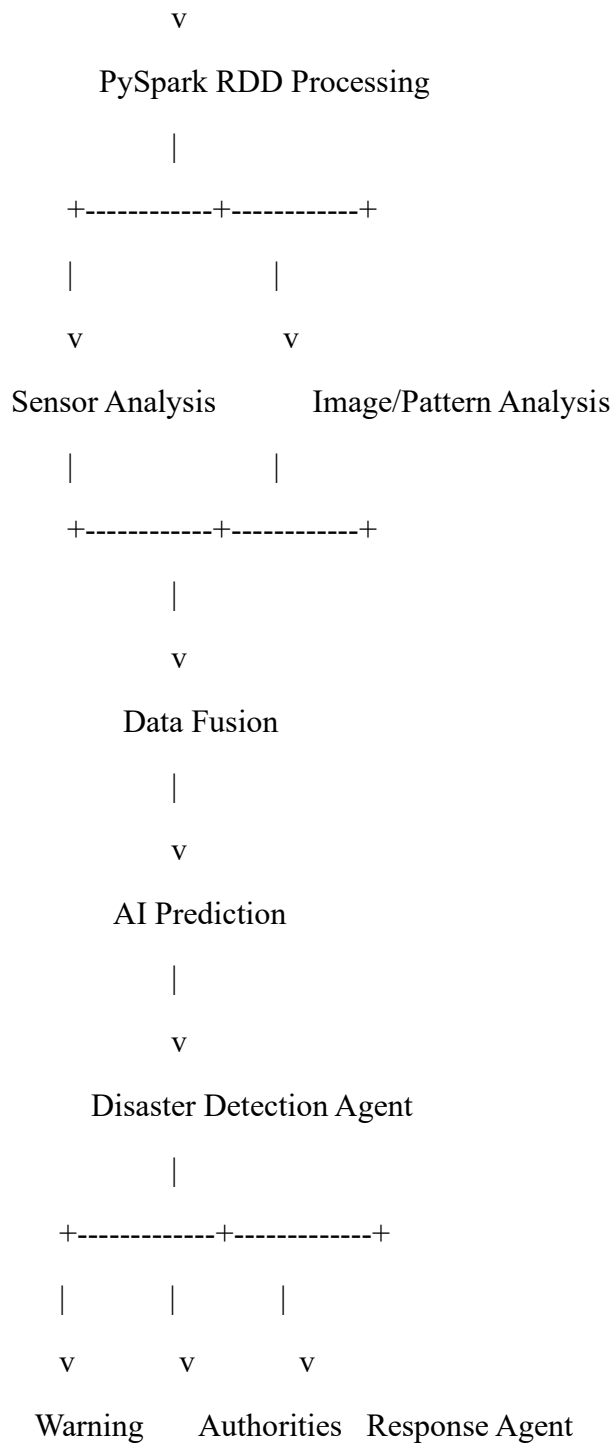
PySpark provides distributed processing for large volumes of incoming data, while intelligent agents coordinate detection, warning and response.

2.2 Objectives

- Detect disasters at an early stage.
- Process multiple data sources simultaneously.
- Combine sensor and satellite information.
- Detect abnormal environmental patterns.
- Generate early warnings.
- Coordinate emergency response.

2.3 Architecture





2.4 Data Sources

IoT Sensors

- Water level
- Temperature
- Pressure
- Seismic activity

- Rainfall
- Soil moisture

Satellite Data

- Flooded areas
- Cloud patterns
- Land changes
- Water expansion

Social Media

Posts may contain:

- Disaster reports
- Location information
- Emergency messages
- Images/videos

2.5 RDD Processing

Sensor data can be represented as:

(sensor_id, location, rainfall, water_level, timestamp)

RDD creation:

```
sensor_rdd = sc.textFile("sensor_data.csv")
```

Parsing:

```
records = sensor_rdd.map(lambda x: x.split(","))
```

Filtering abnormal sensor readings:

```
valid = records.filter(lambda x: float(x[3]) >= 0)
```

Grouping by location:

```
location_data = valid.map(
    lambda x: (x[1], float(x[3]))
).reduceByKey(lambda a, b: max(a, b))
```

2.6 Data Fusion

Different sources can be combined.

For example:

Sensor Data

+

Satellite Data

+

Weather Data

+

Social Media

↓

Data Fusion

↓

Disaster Risk Score

A risk score may consider:

Risk =

$0.30 \times \text{Sensor Risk}$

$+ 0.30 \times \text{Satellite Risk}$

$+ 0.20 \times \text{Weather Risk}$

$+ 0.20 \times \text{Social Signal}$

2.7 Intelligent Agents

Sensor Agent

Monitors sensor information.

Disaster Detection Agent

Identifies abnormal patterns.

Verification Agent

Checks whether multiple sources support the same event.

Warning Agent

Generates warnings.

Emergency Response Agent

Coordinates emergency services.

Communication Agent

Sends notifications to citizens and authorities.

2.8 Example Flood Detection

Suppose the system detects:

Heavy rainfall

+

Rapid increase in river level

+

Satellite water expansion

+

Multiple local emergency reports

The combined evidence produces a high flood-risk score.

The disaster detection agent informs the warning agent.

The warning agent can issue:

HIGH FLOOD RISK

Area: District X

Expected Risk: Severe

Recommended Action: Evacuate low-lying regions

2.9 Early Warning Workflow

Collect

↓

Clean

↓

Process

↓

Fuse

↓

Detect



Verify



Predict



Warn



Respond



Monitor

2.10 Advantages

- Early disaster detection.
- Multi-source intelligence.
- Distributed processing.
- Real-time monitoring.
- Reduced false alarms through data fusion.
- Automated emergency coordination.

2.11 Conclusion

The Smart Disaster Detection System combines PySpark-based distributed processing with multi-source data fusion. Intelligent agents independently monitor, verify and respond to disaster conditions. This architecture enables scalable and timely warnings and can support authorities during emergency situations.

3. AI-BASED FRAUD DETECTION FRAMEWORK

3.1 Introduction

Financial institutions process millions of transactions every day. Fraudulent transactions must be identified quickly to prevent financial losses.

The proposed framework uses PySpark RDDs for distributed transaction processing and intelligent agents for fraud detection, alert generation and continuous learning.

3.2 Objectives

- Process large transaction volumes.
- Detect fraudulent behaviour quickly.
- Identify abnormal transaction patterns.
- Generate automatic alerts.
- Continuously improve fraud detection.
- Reduce false positives.

3.3 Architecture

Financial Transactions

|
v

Data Collection

|
v

PySpark

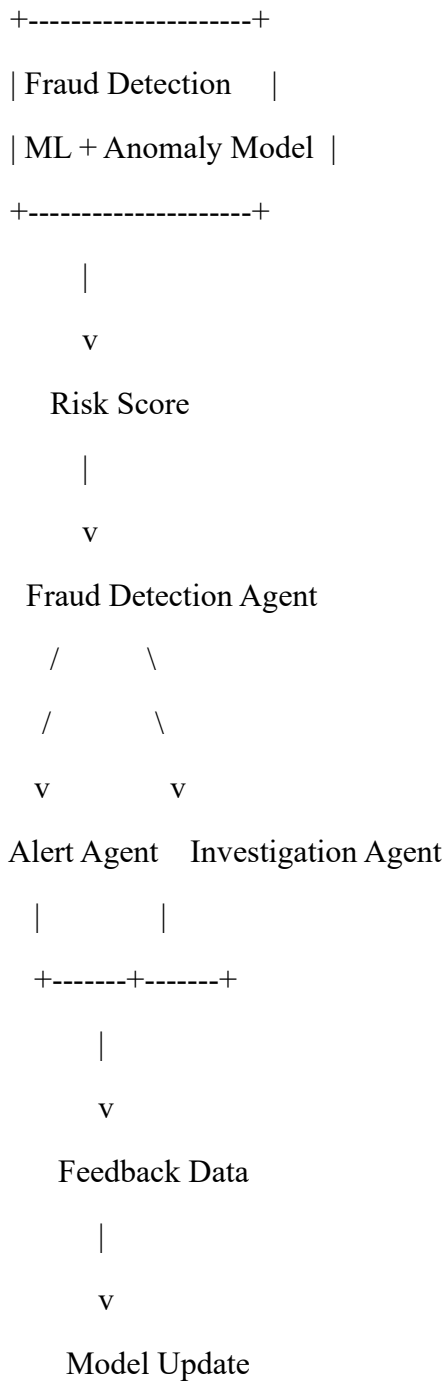
|
v

RDD Processing

|
v

Feature Extraction

|
v



3.4 Transaction Data

Possible fields:

- Transaction ID
- Account ID
- Amount
- Merchant
- Timestamp

- Location
- Device ID
- Transaction type
- Previous transaction history

3.5 RDD Processing

```
transactions = sc.textFile("transactions.csv")
```

```
records = transactions.map(
    lambda x: x.split(",")
)
```

```
valid = records.filter(
    lambda x: len(x) >= 5
)
```

Transaction amount can be extracted:

```
amounts = valid.map(
    lambda x: (x[1], float(x[2]))
)
```

Total transaction value per account:

```
account_total = amounts.reduceByKey(
    lambda a, b: a + b
)
```

3.6 Feature Engineering

Features include:

Transaction Amount

Large deviations from normal spending may indicate fraud.

Transaction Velocity

Many transactions in a short period may indicate suspicious behaviour.

Location

An unexpected location can increase risk.

Device

Transactions from unknown devices may be suspicious.

Merchant

Unusual merchant categories can contribute to the risk score.

3.7 Anomaly Detection

The system can calculate an anomaly score:

Anomaly Score =

Amount Deviation

+

Transaction Velocity

+

Location Deviation

+

Device Risk

Machine learning models such as:

- Isolation Forest
- Random Forest
- Logistic Regression
- Clustering

3.8 Intelligent Agents

Agent	Function
Transaction Agent	Monitors incoming transactions
Feature Agent	Extracts risk features
Detection Agent	Detects suspicious behaviour
Alert Agent	Generates alerts
Investigation Agent	Examines suspicious accounts
Learning Agent	Uses feedback to improve models

3.9 Risk-Based Decision

Example:

Risk Score < 0.30



Approve

0.30–0.70



Additional Authentication

Risk Score > 0.70



Hold / Alert / Investigation

3.10 Continuous Learning

After an alert is investigated:

Transaction



Fraud Detection



Alert



Human Investigation



Fraud / Genuine Label



Training Dataset



Model Update

This allows the system to learn from newly discovered fraud patterns.

3.11 Scalability

PySpark distributes transactions among cluster nodes.

For example:

1 Million Transactions

↓

Partition

↓

Node 1 → 250K

Node 2 → 250K

Node 3 → 250K

Node 4 → 250K

All nodes process their partitions in parallel.

3.12 Conclusion

The proposed fraud detection framework uses PySpark RDDs to process large transaction datasets efficiently. Distributed feature engineering and anomaly detection provide scalability, while intelligent agents coordinate detection, alerts, investigation and learning. The feedback loop continuously improves the system.

4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM

4.1 Introduction

Supply chains involve manufacturers, warehouses, suppliers, transportation providers and customers. Managing inventory, transportation and demand manually can result in excess stock, shortages and increased costs.

An autonomous supply chain optimization system uses distributed AI and intelligent agents to make decentralized decisions.

4.2 Objectives

- Forecast product demand.
- Optimize inventory levels.
- Reduce transportation costs.
- Prevent stockouts.
- Improve delivery efficiency.

- Coordinate suppliers and warehouses.

4.3 Architecture

Customers

|

v

Sales / Orders

|

v

+-----+

| Data Collection |

+-----+

|

v

+-----+

| PySpark RDD Processing|

+-----+

|

+-----+-----+

|

|

|

v

v

v

Demand Inventory Logistics

Forecasting Optimization Optimization

|

|

|

+-----+-----+

|

v

Intelligent Agents

/

|

\

v

v

v

Supplier Warehouse Transport

Agent Agent Agent

\ | /

\ | /

v v v

Coordinator

|

v

Optimized Decisions

4.4 Data Sources

- Historical sales
- Current orders
- Inventory levels
- Supplier information
- Delivery times
- Transportation costs
- Warehouse capacity
- Seasonal trends
- Customer demand

4.5 RDD Transformations

Sales data can be represented as:

(product_id, quantity, date)

Create an RDD:

```
sales = sc.textFile("sales.csv")
```

Parse:

```
records = sales.map(lambda x: x.split(","))
```

Group sales by product:

```
product_sales = records.map(
```

```
lambda x: (x[0], int(x[1])))
```

).reduceByKey(lambda a, b: a + b)

The result gives total sales for each product.

4.6 Demand Forecasting

Historical demand can be analysed to predict future demand.

For example:

Previous Demand



Seasonal Pattern



Recent Sales



AI Forecast



Expected Demand

If predicted demand increases, inventory requirements can automatically increase.

4.7 Intelligent Agents

Supplier Agent

Determines when to reorder materials.

Inventory Agent

Maintains optimal stock levels.

Warehouse Agent

Allocates products among warehouses.

Transportation Agent

Selects efficient routes and carriers.

Demand Agent

Forecasts future customer demand.

Coordination Agent

Coordinates all agents.

4.8 Decentralized Decision-Making

Example:

Warehouse A:

Stock = 100

Expected Demand = 300

The inventory agent identifies a shortage.

It communicates with the supplier agent:

Required Quantity = 200

The supplier agent checks availability.

The transportation agent calculates delivery cost and route.

The coordinator chooses the most cost-effective option.

4.9 Cost Optimization

The system attempts to minimize:

Total Cost =

Inventory Cost

+ Transportation Cost

+ Storage Cost

+ Stockout Cost

+ Ordering Cost

The agents continuously evaluate these factors.

4.10 Example

Suppose demand forecast shows:

Current Stock = 500

Expected Demand = 800

Safety Stock = 100

Required inventory:

Required Stock = $800 + 100 = 900$

Additional quantity:

$900 - 500 = 400$

The inventory agent requests 400 additional units.

4.11 Benefits

- Lower inventory cost.
- Reduced stockouts.
- Faster decision-making.
- Efficient transportation.
- Better demand forecasting.
- Autonomous operation.
- Scalable processing.

4.12 Conclusion

The autonomous supply chain system uses PySpark RDDs to process large-scale sales, inventory and logistics data. Multiple intelligent agents collaborate to make decentralized decisions. The system continuously balances inventory, demand and transportation costs, resulting in an efficient and adaptive supply chain.

5. PERSONALIZED LEARNING ANALYTICS PLATFORM

5.1 Introduction

Traditional educational systems provide the same learning content to every student. However, students have different learning speeds, strengths and weaknesses.

A Personalized Learning Analytics Platform uses AI to analyse student behaviour and automatically adapt learning content.

PySpark provides scalable processing for thousands of learners, while intelligent agents personalize content and provide real-time feedback.

5.2 Objectives

- Analyse student learning behaviour.
- Identify knowledge gaps.
- Measure learning performance.
- Recommend personalized content.
- Adjust difficulty dynamically.
- Provide real-time feedback.
- Support thousands of learners.

5.3 Architecture

STUDENTS

|

+-----+-----+

|

|

Quiz Data

Behaviour Data

|

|

+-----+-----+

|

v

Data Collection Layer

|

v

PySpark / RDD Layer

|

v

Feature Engineering

|

v

Learning Analytics

|

+-----+-----+

|

|

v

v

Performance

Engagement

Analysis

Analysis

|

|

+-----+-----+

|

v

Learning Agent

|

v

Personalized Content

|

v

Student

|

v

Feedback

|

+-----> Model Update

5.4 Student Data

The platform can collect:

- Student ID
- Course
- Topic
- Quiz marks
- Number of attempts
- Time spent
- Questions answered
- Hint usage
- Video completion
- Assignment scores
- Feedback

5.5 RDD Processing

Student data:

(student_id, topic, score)

RDD:

```
students = sc.textFile("student_data.csv")
```

Parsing:

```
records = students.map(  
    lambda x: x.split(",")  
)
```

Student-topic score:

```
scores = records.map(  
    lambda x: ((x[0], x[1]), float(x[2])))  
)
```

Average score:

```
avg_scores = scores.combineByKey(  
    lambda x: (x, 1),  
    lambda acc, x: (acc[0] + x, acc[1] + 1),  
    lambda a, b: (a[0] + b[0], a[1] + b[1])  
)mapValues(lambda x: x[0] / x[1])
```

5.6 Learning Analytics

The system calculates:

Accuracy

Accuracy =

Correct Answers / Total Answers

Engagement

Engagement =

Time Spent + Activities Completed

Mastery

A topic can be classified as:

0–40% → Weak

40–70% → Developing

70–85% → Good

85–100% → Mastered

5.7 Knowledge Gap Detection

Example:

Topic	Score	Status
Python	90%	Mastered
Statistics	65%	Developing
Machine Learning	35%	Weak
SQL	82%	Good

The system identifies **Machine Learning** as the primary knowledge gap.

5.8 Intelligent Agents

Agent	Responsibility
Learner Agent	Maintains learner profile
Assessment Agent	Analyses performance
Content Agent	Selects learning material
Difficulty Agent	Adjusts question difficulty
Feedback Agent	Provides feedback
Progress Agent	Tracks improvement

5.9 Adaptive Learning

Suppose a student repeatedly performs poorly on a topic.

The difficulty agent can change:

Advanced Question



Intermediate Question



Basic Question



Concept Explanation



Practice



Assessment

If performance improves:

Basic



Intermediate



Advanced

Thus, learning becomes personalized.

5.10 Real-Time Feedback

After a quiz:

Student submits quiz



RDD Processing



Performance Analysis



Knowledge Gap Detection



Learning Agent



Personalized Recommendation

Example:

Your performance in Probability is 42%.

Recommended:

1. Review conditional probability.
2. Watch the concept explanation.
3. Attempt 5 basic questions.
4. Take the assessment again.

5.11 Scalability

Suppose the platform has 100,000 students.

Instead of processing all data on one machine:

100,000 Students



RDD Partitioning



Node 1 → Students 1–20,000

Node 2 → Students 20,001–40,000

Node 3 → Students 40,001–60,000

Node 4 → Students 60,001–80,000

Node 5 → Students 80,001–100,000

Processing occurs in parallel.

5.12 Feedback Loop

Observe



Analyse



Identify Knowledge Gap



Recommend



Student Learns



Assess



Collect Feedback



Update Learner Model

5.13 Advantages

- Personalized learning.
- Automated knowledge-gap identification.
- Real-time feedback.
- Adaptive difficulty.
- Large-scale student analytics.
- Continuous improvement.
- Better student engagement.

5.14 Conclusion

The Personalized Learning Analytics Platform combines PySpark RDDs with intelligent agents to process large-scale educational data. Learning agents analyse student performance and dynamically adjust content difficulty. The feedback loop allows the platform to continuously learn from student behaviour and deliver personalized educational experiences.